Ex 6 – Semantic Movie Recommendation System using Vector Database

Learning Objective

Implement a scalable semantic search engine that utilizes high-dimensional vector embeddings and a vector database (**FAISS**) to retrieve contextually relevant media from a large-scale dataset.

Question

Consider you are a Lead Data Engineer at CineMatch, a rapidly growing streaming platform. The current search system uses **Keyword Matching**, which means if a user searches for *"movies about space exploration,"* they only get results if the exact words like *"space"* or *"exploration"* appear in the title.

The Marketing Team reports that users are frustrated because they cannot search based on **themes, emotions, or concepts**. For example:

- Searching *"heartwarming stories"* does not return movies like *Toy Story*
- Searching *"AI and robots"* may miss relevant movies without those exact keywords

Your objective is to design a **Semantic Search Prototype** using the MovieLens dataset that understands **meaning instead of exact words**.

Your Solution Must

- Convert movie titles and genres into **high-dimensional vector embeddings** so that semantically similar words (e.g., *Galaxy* and *Space*) are close in vector space
- Use **FAISS** for efficient similarity search across large datasets
- Ensure fast retrieval even when the dataset scales to thousands or millions of movies
- Combine multiple features (Title + Genres) into a single representation for better context understanding
- Use cosine similarity (or equivalent) to measure closeness between movies and user queries

Tasks to Perform

1. Install required libraries and set up the environment for embedding generation and similarity search
2. Download the MovieLens dataset and load the movie data into a structured format
3. Clean and preprocess the dataset by handling missing values and formatting text fields

4. Combine relevant features such as **Title and Genres** into a single searchable text representation
5. Generate high-dimensional embeddings for each movie using a pre-trained embedding model
6. Store all embeddings in a **FAISS** index for efficient similarity search
7. Maintain metadata (movie ID, title, genres) linked to each embedding
8. Accept a natural language query from the user (e.g., *"a movie about friendship and adventure"*)
9. Convert the user query into an embedding using the same model
10. Perform similarity search in FAISS to retrieve the top relevant movies
11. Extract and display the matched movie titles along with similarity scores
12. Evaluate the system by comparing keyword-based results vs semantic search results

Learning Outcome

Upon completion of this program, students will be able to:

- Understand the limitations of traditional keyword-based search systems
- Transform unstructured text into **high-dimensional vector embeddings**
- Use **FAISS** to perform fast similarity-based retrieval
- Design a **semantic search pipeline** that captures meaning and context
- Apply vector search techniques to real-world applications like recommendation systems and search engines

Program

```
import pandas as pd

import numpy as np

import faiss

from sentence_transformers import SentenceTransformer


df = pd.read_csv("/content/movies.csv")

print(df)

df = df.dropna()
```

```
df["genres"] = df["genres"].str.replace("|", " ")

df["combined"] = df["title"] + " " + df["genres"]

model = SentenceTransformer("all-MiniLM-L6-v2")

embeddings = model.encode(df["combined"].tolist(), show_progress_bar=True)

embeddings = np.array(embeddings).astype("float32")

dimension = embeddings.shape[1]

faiss.normalize_L2(embeddings)

index = faiss.IndexFlatIP(dimension)
index.add(embeddings)

print(f"Indexed {index.ntotal} movies.")

metadata = df[["id", "title", "genres"]].reset_index(drop=True)

query = input("Enter your search query: ")

query_embedding = model.encode([query])
query_embedding = np.array(query_embedding).astype("float32")

faiss.normalize_L2(query_embedding)

k = 5
scores, indices = index.search(query_embedding, k)
```

```
print("\n🔍 Top Semantic Results:\n")

for i, idx in enumerate(indices[0]):
    print(f"{i+1}. {metadata.iloc[idx]['title']} ({metadata.iloc[idx]['genres']})")
    print(f"   Similarity Score: {scores[0][i]:.4f}\n")

print("\n🔎 Keyword-Based Results:\n")

keyword_results = df[df["combined"].str.contains(query, case=False, na=False)].head(5)

for i, row in keyword_results.iterrows():
    print(f"- {row['title']} ({row['genres']})")
```

Output:

🔍 **Top Semantic Results:**

1. Alien (Horror Action Thriller Science Fiction)
   Similarity Score: 0.6964

2. Aliens in the Attic (Adventure Comedy Family Fantasy Science Fiction)
   Similarity Score: 0.6384

3. Aliens vs Predator: Requiem (Fantasy Action Science Fiction Thriller Horror)
   Similarity Score: 0.6160

4. Interstellar (Adventure Drama Science Fiction)
   Similarity Score: 0.5201

5. Monsters vs Aliens (Animation Family Adventure Science Fiction)

Similarity Score: 0.4878